

HelixConstitution

HelixConstitution

सार्वभौमिक इंजीनियरिंग संविधान जिसे हर परियोजना विरासत में पाती है — धोखाधड़ी-विरोधी कानून, यांत्रिक रूप से लागू, एक Git सबमॉड्यूल के रूप में साझा।

सारांश

HelixConstitution एकमात्र, परियोजना-निरपेक्ष नियमपुस्तिका है — जिसे हर Helix/basic-digital परियोजना द्वारा Git सबमॉड्यूल के रूप में जोड़ा जाता है — जो अटल इंजीनियरिंग अनुशासन (धोखाधड़ी-विरोधी, साक्ष्य-आधारित सत्यापन, डेटा/होस्ट सुरक्षा, दस्तावेजीकरण और परीक्षण कवरेज) को कोडित करती है और इसे 140 से अधिक रिपॉजिटरीज़ के समूह में प्रसारित करती है। यह वह शासन-रीढ़ है जो पूरे परिवार को एकसूत्र में बाँधती है।

संक्षिप्त विवरण

एक सार्वभौमिक, विरासत में मिलने योग्य Constitution, जो Git सबमॉड्यूल के रूप में वितरित की जाती है। यह अनिवार्य, अटल नियमों को परिभाषित करती है — धोखाधड़ी-विरोधी साक्ष्य जाँच, गलत-सकारात्मक प्रतिरक्षा, डेटा और होस्ट सुरक्षा, कवरेज और दस्तावेजीकरण अनुशासन — जिन्हें हर उपभोग करने वाली परियोजना स्वतः विरासत में पाती है और विस्तारित कर सकती है, पर कमज़ोर नहीं कर सकती।

विस्तृत विवरण

HelixConstitution उन इंजीनियरिंग प्रथाओं के लिए प्रामाणिक, एकमात्र सत्य-स्रोत है जो हर उस परियोजना द्वारा साझा की जाती हैं जो इसे Git सबमॉड्यूल के रूप में जोड़कर अपनाती है — इंजीनियरिंग कानून, कोड की तरह ही वितरित और संस्करण-बद्ध। इसका केंद्रबिंदु — Constitution.md — एक लगभग 1 MB का, निरंतर-संस्करणित दस्तावेज़ है जिसमें क्रमांकित धाराएँ (\$11.4.x प्रतिज्ञा परिवार, वर्तमान में \$11.4.170 तक) और प्रति-एजेंट संचालन नियमावली (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md) शामिल हैं, जो इसे संदर्भ द्वारा आयात करती हैं ताकि मनुष्य और हर CLI एजेंट एक ही नियमपुस्तिका से पढ़ें। विरासत जानबूझकर तीन स्तरों की है: सार्वभौमिक आधार (यह सबमॉड्यूल), परियोजना स्तर (परियोजना का अपना Constitution/CLAUDE/AGENTS जो इसे विस्तारित करता है), और एक वैकल्पिक प्रति-सबडायरेक्टरी स्तर — ऊपर से नीचे तक मूल्यांकित, जहाँ परियोजना नियमों को कड़ा कर सकती है, पर उन्हें

कमज़ोर करने की संरचनात्मक मनाही है। परिणामस्वरूप 140 से अधिक रिपॉजिटरीज़ का एक ऐसा समूह बनता है जो चुपचाप अलग नहीं हो सकता, क्योंकि उनके द्वारा साझा किया गया अनुशासन पिन किया हुआ है, याद नहीं रखा गया।

यह दस्तावेज़ अडिग रूप से डोमेन-निरपेक्ष है: किसी भी विक्रेता, हार्डवेयर SKU, पोर्ट या लाइब्रेरी संस्करण का नाम लेने वाली कोई भी बात उपभोग करने वाली परियोजना के अपने Constitution में स्थानांतरित होनी चाहिए, और सार्वभौमिकता कभी मान ली नहीं जाती — इसे आधार में नियम शामिल करने से पहले एक स्पष्ट चार-भागीय परीक्षण के विरुद्ध सिद्ध करना होता है। इसकी दार्शनिक रीढ़ धोखाधड़ी-विरोधी है, जो आपस में जुड़े प्रतिज्ञा परिवारों के रूप में व्यक्त की गई है — §1.1 गलत-सकारात्मक प्रतिरक्षा, §11.4 अंतिम-उपयोगकर्ता गुणवत्ता प्रतिज्ञा, §11.4.6 अनुमान-निषेध, §11.4.69 सकारात्मक-साक्ष्य वर्गीकरण — जिनका संयुक्त प्रभाव एक ही कठोर रेखा है: शिपिंग का मानदंड कभी “परीक्षण पास” नहीं होता, बल्कि यह होता है कि “एक वास्तविक उपयोगकर्ता इस सुविधा का उपयोग कर सकता है,” और हर हरे परिणाम को पकड़े गए भौतिक साक्ष्य का हवाला देना चाहिए, अन्यथा वह गिनती में नहीं आता। एक सहयोगी submodules-catalogue.md (142 रिपॉज) “क्या हमारे पास पहले से ऐसा कुछ है जो यह काम करता है?” को एक कैटलॉग-प्रथम, विस्तारित-न-करें-फिर-से-लागू दृष्टिकोण में बदल देता है, इससे पहले कि एक भी नई पंक्ति कोड लिखी जाए। सहायक स्क्रिप्ट किसी भी नेस्टिंग गहराई से सबमॉड्यूल का पता लगाती हैं और हर कमिट को चार स्वतंत्र Git प्रदाताओं तक प्रसारित करती हैं, ताकि एकमात्र प्रामाणिक नियमपुस्तिका खोना भी असंभव हो।

सामग्री

हमने इसे क्यों बनाया

एक ही स्वामी द्वारा लिखे गए कई बड़े उत्पाद ऐप्स और दर्जनों अलग-अलग पुनःप्रयोगी सबमॉड्यूल्स बार-बार उन्हीं कठिनाई से प्राप्त नियमों को फिर से खोजते रहे—और बार-बार उसी विफलता वर्ग से टकराते रहे: टेस्ट और स्टेटस रिपोर्ट्स जो सफलता का दावा करते हुए अंतिम उपयोगकर्ता के लिए फीचर को टूटा हुआ छोड़ देते हैं (“PASS-ब्लफ़” और “FAIL-ब्लफ़”)। Constitution के प्रत्येक फॉरेंसिक एंकर में एक वास्तविक घटना दर्ज है (जैसे 2026-05-20 का D3 ऑडियो-रूटिंग PASS-ब्लफ़, जहाँ “कोडेक इन यूज़” फ़ील्ड खाली होने के बावजूद वैलिडेशन हरा हो गया, या 2026-06-25 का विशाल-बटन UI जो टोकन-इक्वैलिटी टेस्ट पास कर गया जबकि असली स्क्रीन टूटी हुई थी)। Constitution का अस्तित्व इसी पूरे वर्ग के बेईमान सफलता को यांत्रिक रूप से असंभव बनाने के लिए है—एक बार, सार्वभौमिक रूप से—ताकि यह अनुशासन परियोजनाओं के बीच भटक न सके या चुपचाप भुला न दिया जाए।

यह क्यों गेम-चेंजर है

यह इंजीनियरिंग संस्कृति को दस्तावेज़ीकरण-जिसे-लोगों-से-आशा-है-कि-पालन-करेंगे से बदलकर विरासत में मिले, वर्जनयुक्त, यांत्रिक रूप से लागू कानून में तब्दील कर देता है—जैसे शैली गाइड और कंपाइलर के बीच का अंतर। एक सबमॉड्यूल अपग्रेड पूरे बेड़े के नियमों को एक साथ, परमाणु रूप से और ट्रेसेबल तरीके से अपडेट कर देता है। एक एंटी-ब्लफ़ अनुबंध गारंटी के साथ हर उपभोगी रेपो में मौजूद होता है, भरोसे पर नहीं बल्कि निर्माण द्वारा: एक प्रोपेगेशन गेट पूरे बेड़े में क्लॉज़ नंबर को शाब्दिक रूप से खोजता है, और एक संबद्ध म्यूटेशन टेस्ट साबित करता है कि गेट खुद ब्लफ़ नहीं कर रहा—इसलिए लागू

करने का तरीका भी लागू होता है। गवर्नेंस विकी पर लिखी गई एक आकांक्षा नहीं रह जाती, बल्कि एक ऑडिटेबल, टेस्टेबल तथ्य बन जाती है जिसे आप CI जॉब की ओर इशारा कर सकते हैं।

क्या है नवाचारी

- **Constitution-एज-सबमॉड्यूल** — इंजीनियरिंग कानून को ठीक कोड की तरह वितरित और वर्जन-पिन किया जाता है, जिसमें जानबूझकर v1.0.0 शैली के टैग और प्रति-परियोजना पिनंग होती है, ताकि हर रेपो को ठीक-ठीक पता हो कि वह किस संशोधन के कानून से बंधा है।
- **एंटी-ब्लफ़ को प्रथम श्रेणी की फॉरेंसिक सिद्धांत के रूप में** — हर क्लॉज़ एक शब्दशः ऑपरेटर आदेश और अक्सर उस वास्तविक घटना से जुड़ा होता है जिसने उसे प्रेरित किया, इसलिए नियम-पुस्तिका राय के बजाय केस लॉ की तरह पढ़ी जाती है।
- **नियमों का मेटा-टेस्टिंग (§1.1)** — हर गेट के साथ एक म्यूटेशन जुड़ा होता है जिसे PASS→FAIL में बदलना ही होता है, इसलिए “गेट एक धोखा नहीं है” केवल दावा नहीं बल्कि हर रन पर साबित होता है; एक गेट जो कभी फेल नहीं हो सकता, उसे गेट न होने से भी बदतर माना जाता है।
- **अर्जित सार्वभौमिकता** — एक स्पष्ट चार-भागीय टेस्ट तय करता है कि कोई नियम वास्तव में सार्वभौमिक है या केवल परियोजना-विशिष्ट, ताकि आधार दुबला, पोर्टेबल और वेंडर लीकेज से मुक्त रहे।

सभी उत्पादों में इसका उपयोग कैसे होता है (इसकी शक्तियाँ)

एक अनिवार्य गवर्नेंस स्तंभ के रूप में, HelixConstitution वह दस्तावेज़ नहीं है जिसका परिवार परामर्श करता है—बल्कि वह भार वहन करने वाली संरचना है जिस पर परिवार खड़ा है:

- **गवर्नेंस की रीढ़:** हर Helix/वैसिक-डिजिटल परियोजना इसे एक सबमॉड्यूल के रूप में जोड़ती है और CLAUDE.md / AGENTS.md / QWEN.md या अपने Constitution.md से आयात करती है; नियम बिना शर्त लागू होते हैं, पहले कमिट से ही, बिना किसी परियोजना-विशेष छूट के।
- **गेट्स और आदेश:** यह चार-स्तरीय कवरेज मॉडल को परिभाषित करता है—स्रोत मौजूद, बिल्ड में टिका रहे, रनटाइम पर व्यवहार करे, गेट ब्लफ़ न हो—जिसे एक फीचर को सभी चार स्तरों पर पार करना होता है तभी वह पूरा माना जाता है, साथ ही नामित आदेशों की एक बढ़ती सूची: क्रेडेंशियल हैंडलिंग (§11.4.10), हमेशा-सिंक दस्तावेज़ीकरण (§11.4.60), कंटेनर्स-सबमॉड्यूल आदेश (§11.4.76), कोडग्राफ (§11.4.78), अनिवार्य टेस्ट-टाइप कवरेज (§11.4.169), और अन्य।
- **प्रसार:** CM-COVENANT-114-NNN-PROPAGATION गेट्स यह सुनिश्चित करते हैं कि शाब्दिक क्लॉज़ टेक्स्ट उपभोगी बेड़े में मौजूद हो, ताकि अनुबंध को एस्टेट के किसी कोने में चुपचाप हटाया न जा सके, गैर-अनुपालन एक कठोर रिलीज़ ब्लॉकर है जिसमें कोई बचाव-रास्ता नहीं होता।
- **खोज:** submodules-catalogue.md “क्या हमारे पास पहले से ही ऐसा कुछ है जो X करता है?” का उत्तर एक नज़र में दे देता है, इससे पहले कि कोई नया मॉड्यूल बनाया जाए, जिससे डुप्लीकेट प्रयास जड़ पर ही खत्म हो जाता है।
- **AI एजेंट्स में एकरूपता:** वही कानून हर CLI एजेंट (Claude कोड, कोडेक्स/कर्सर/एडर/ओपनकोड/क्रश/किमी via AGENTS.md, Qwen कोड via QWEN.md)

को समान रूप से व्यक्त किया जाता है, ताकि कोई भी टूल कोड को छुए, वह एक ही अनुबंध का पालन करे।

सामग्री

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **मनमाने गहराई वाले नेस्टेड सबमॉड्यूल का पता लगाना** — तीन सबमॉड्यूल गहराई में दबी कोई नियमावली हो, उसे कानून का पता लगाना ही होता है, भले ही वह कहाँ स्थित हो → `find_constitution.sh` पैरेंट डायरेक्टरी में ऊपर की ओर चलता है और गिट सुपरप्रोजेक्ट पॉइंटर का पुनरावर्ती रूप से पालन करता है, `CONSTITUTION_DIR` ओवरराइड का सम्मान करता है और दो समर्थित लेआउट (`constitution/`, `submodules/constitution/`) का उपयोग करता है, ताकि नेस्टिंग कितनी भी गहरी क्यों न हो, समाधान हमेशा निर्धारक हो।
- **चार गिट प्रदाताओं में एक रिपॉजिटरी को प्राधिकृत बनाए रखना** — यदि मिरर ड्रिफ्ट हो जाएँ तो वे बेकार हो जाते हैं → `install_upstreams.sh` घोषणात्मक `Upstreams/*.sh` रिमोट्स को पढ़ता है और `origin` को कई पुश URL के साथ कॉन्फ़िगर करता है, ताकि एक ही `git push` GitHub (प्राथमिक), GitLab, GitFlic और GitVerse पर परमाणु रूप से प्रसारित हो जाए और कोई भी मिरर पीछे न रह जाए।
- **सार्वभौमिक आधार में नियमों की अतिवृद्धि/परियोजना रिसाव रोकना** — हर “इसे यहीं जोड़ दो” का प्रलोभन पोर्टेबिलिटी को कमजोर करता है → अर्जित-सार्वभौमिकता का चार-भागीय परीक्षण और §11.4.17 सार्वभौमिक-बनाम-परियोजना वर्गीकरण प्रत्येक नए नियम पर लागू किया जाता है, जिससे परियोजना-विशिष्ट चिंताओं को वापस परियोजना परत में धकेल दिया जाता है जहाँ उनका स्थान है।
- **विरासत गेट की कार्यक्षमता साबित करना** — एक ऐसा गेट जिसकी विफलता कभी दिखाई न दे, वह गेट भरोसेमंद नहीं होता → `meta_test_inheritance.sh`, एक प्रहरी मेटा-टेस्ट, जानबूझकर §11.4 एंकर को हटा देता है और दावा करता है कि गेट उसे पकड़ लेता है, ताकि प्रवर्तन तंत्र स्वयं ही चुपचाप टूटने के खिलाफ निरंतर पुनःसत्यापित होता रहे।

तकनीकी ढाँचा

- **गिट-सबमॉड्यूल विरासत** — क्यों: गिट सबमॉड्यूल ही एकमात्र ऐसा तंत्र है जो किसी नियमपुस्तिका को प्राधिकृत और उपभोक्ता के अनुसार संस्करण-बद्ध बनाए रखने की अनुमति देता है, जिसे स्पष्ट और समीक्षा योग्य अपग्रेड के माध्यम से निष्पादित किया जाता है, न कि चुपचाप कॉपी-पेस्ट करके; कैसे: उपभोग करने वाली परियोजनाएँ सबमॉड्यूल जोड़ती हैं और उसकी एजेंट फ़ाइलों को `@import` करती हैं, और तीनों परतों का मूल्यांकन ऊपर-से-नीचे किया जाता है, जहाँ हर सीमा पर “विस्तारित करता है, कमजोर नहीं” का कड़ा अनुबंध लागू होता है।
- **`find_constitution.sh`** — क्यों: नियम तब बेकार हो जाते हैं जब गहराई से नेस्टेड कोड उन्हें विश्वसनीय रूप से नहीं खोज पाता, और पथ हार्डकोड करने से परियोजना के पुनर्गठन के क्षण ही टूट जाएगी; कैसे: पैरेंट-डायरेक्टरी वाक और `git rev-parse --show-superproject-working-tree` पुनरावर्तन, `CONSTITUTION_DIR` ओवरराइड द्वारा समर्थित, दोनों समर्थित लेआउट का समाधान करता है।
- **`install_upstreams.sh` + `Upstreams/`** — क्यों: चार प्रदाताओं की अतिरेक तभी वास्तविक होती है जब इसे बनाए रखने में शून्य अतिरिक्त प्रयास लगे, अन्यथा

मिरर सड़ जाते हैं; कैसे: प्रति-रिमोट घोषणात्मक .sh फ़ाइलें एकल मल्टी-URL origin में परिवर्तित की जाती हैं, जिससे चार पुश एक में समाहित हो जाते हैं।

- **§1.1 उत्परिवर्तन मेटा-टेस्ट** — क्यों: एक ऐसा गेट जो कभी विफल न हो, वह न होने से भी बदतर है क्योंकि यह झूठा आत्मविश्वास पैदा करता है; कैसे: प्रत्येक गेट के साथ एक sed-आउट/रिनेम उत्परिवर्तन जोड़ा जाता है जो PASS→FAIL में बदलना चाहिए और फिर पुनर्स्थापित किया जाता है, ताकि हर गेट हर रन पर यह साबित करे कि वह अभी भी प्रभावी है।
- **प्रसार गेट (CM-COVENANT-114-NNN-PROPAGATION)** — क्यों: एक अनुबंध तभी सार्वभौमिक होता है जब वह प्रत्येक उपभोक्ता में सत्यापित रूप से मौजूद हो, न कि केवल प्रमुख रिपॉजिटरी में; कैसे: उपभोक्ताओं में क्लॉज-नंबर की शाब्दिक ग्रेपिंग, §1.1 उत्परिवर्तन द्वारा समर्थित, जो साबित करता है कि प्रसार जाँच स्वयं विफल हो सकती है।
- **submodules-catalogue.md (§11.4.74)** — क्यों: डुप्लीकेशन-विरोधी अनुशासन का उल्लंघन करने का सबसे तेज़ तरीका यह है कि आपको पता ही न चले कि आपके पास पहले से क्या है; कैसे: 142 रिपॉजिटरी का क्षमता-आधारित सूचीपत्र, जिसमें कुछ भी नया बनाने से पहले सूची-जाँच ट्रैकर में दर्ज की जाती है।
- **बहु-प्रारूप निर्यात** — क्यों: एक ही कानून को मनुष्यों के पढ़ने, टूलिंग द्वारा पार्स करने और अभिलेखागार द्वारा संरक्षित करने के लिए समान रूप से उपभोग्य होना चाहिए; कैसे: प्रत्येक मानक दस्तावेज़ को एक ही स्रोत से .md / .html / .pdf / .docx के रूप में निर्यात किया जाता है।

सामग्री

स्थिति एवं ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** प्रेषित। सक्रिय रूप से संस्करणबद्ध और पुरे बेड़े में उप-मॉड्यूल के रूप में उपयोग में (सार्वजनिक कैनोनिकल और मिरर रिपॉजिटरीज़)।
- **लाइसेंस:** निर्धारित किया जाना शेष — समीक्षित स्रोत सामग्री में स्पष्ट रूप से उल्लेखित नहीं; प्रकाशन से पूर्व रिपॉजिटरी के LICENSE से पुष्टि करें।
- **अतिरिक्त अपस्ट्रीम मिरर:** GitLab helixdevelopment1/helixconstitution, GitFlic helixdevelopment/helixconstitution, GitVerse helixdevelopment/HelixConstitution।

प्राथमिकता स्तर: Helix-प्राथमिक — Helix परिवार में निर्मित हर चीज़ के लिए एक अनिवार्य शासन स्तंभ।